



Файли конфігурації Kubernetes

Як створювати об'єкти

Використовуйте `kubectl apply` для створення всіх об'єктів, окрім тих, які вже існують, визначених файлами конфігурації у вказаному каталозі:

```
kubectl apply -f <directory>/
```

Це встановлює анотацію `kubectl.kubernetes.io/last-applied-configuration: '{...}'` для кожного об'єкта. Анотація містить вміст файлу конфігурації об'єкта, який використовувався для створення об'єкта.

Ось приклад файлу конфігурації об'єкта:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Як створювати об'єкти

Запустіть `kubectl diff`, щоб надрукувати об'єкт, який буде створено:

```
kubectl diff -f https://k8s.io/examples/application/simple_deployment.yaml
```

Створіть об'єкт за допомогою `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Надрукуйте живу конфігурацію за допомогою `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

Як створювати об'єкти

Вихідні дані показують, що анотація

kubectl.kubernetes.io/last-applied-configuration була записана в поточну конфігурацію, і вона відповідає файлу конфігурації:

```
kind: Deployment
metadata:
  annotations:
    # ...
    # This is the json representation of simple_deployment.yaml
    # It was written by kubectl apply when the object was created
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"minReadySeconds":5,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"Labels":{"app":"nginx"}},
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
  # ...
```

Як оновити об'єкти

Ви також можете використовувати `kubectl apply` для оновлення всіх об'єктів, визначених у каталозі, навіть якщо ці об'єкти вже існують. Цей підхід забезпечує наступне:

1. Встановлює поля, які відображаються у файлі конфігурації в поточній конфігурації.
2. Очищає поля, видалені з файлу конфігурації в поточній конфігурації.

```
kubectl diff -f <directory>/  
kubectl apply -f <directory>/
```

Ось приклад файлу конфігурації:



```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-deployment  
spec:  
  selector:  
    matchLabels:  
      app: nginx  
  minReadySeconds: 5  
  template:  
    metadata:  
      labels:  
        app: nginx  
    spec:  
      containers:  
        - name: nginx  
          image: nginx:1.14.2  
          ports:  
            - containerPort: 80
```

Як оновити об'єкти

Створіть об'єкт за допомогою `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Надрукуйте живу конфігурацію за допомогою `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

Як оновити об'єкти

Вихідні дані показують, що анотація

kubectl.kubernetes.io/last-applied-configuration була записана в поточну конфігурацію, і вона відповідає файлу конфігурації:

```
kind: Deployment
metadata:
  annotations:
    # ...
    # This is the json representation of simple_deployment.yaml
    # It was written by kubectl apply when the object was created
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"minReadySeconds":5,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"Labels":{"app":"nginx"}},
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
      # ...
    # ...
  # ...
# ...
```

Як оновити об'єкти

Безпосередньо оновіть поле `реплік` у поточній конфігурації за допомогою `kubectl scale`. Тут не використовується `kubectl apply`:

```
kubectl scale deployment/nginx-deployment --replicas=2
```

Роздрукуйте живу конфігурацію за допомогою `kubectl get`:

```
kubectl get deployment nginx-deployment -o yaml
```


Як оновити об'єкти

Вихідні дані показують, що для поля репліки встановлено значення 2, а анотація останньої застосованої конфігурації не містить поля репліки:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # note that the annotation does not contain replicas
    # because it was not updated through apply
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"minReadySeconds":5,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}}},
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  replicas: 2 # written by scale
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
  spec:
    containers:
      - image: nginx:1.14.2
        # ...
        name: nginx
        ports:
          - containerPort: 80
        # ...
```

Як оновити об'єкти

Оновіть файл конфігурації `simple_deployment.yaml`, щоб змінити зображення з `nginx:1.14.2` на `nginx:1.16.1`, і видаліть поле `minReadySeconds`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.16.1 # update the image
          ports:
            - containerPort: 80
```

Як оновити об'єкти

Застосуйте зміни, внесені до файлу конфігурації:

```
kubectl diff -f https://k8s.io/examples/application/update_deployment.yaml  
kubectl apply -f https://k8s.io/examples/application/update_deployment.yaml
```

Надрукуйте живу конфігурацію за допомогою `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/update_deployment.yaml -o yaml
```

Вихідні дані показують наступні зміни поточної конфігурації:

- Поле копій зберігає значення 2, встановлене шкалою kubectl. Це можливо, тому що його випущено з конфігураційного файлу.
- Поле зображення було оновлено до nginx:1.16.1 з nginx:1.14.2.
- Анотацію останньої застосованої конфігурації оновлено новим зображенням.
- Поле minReadySeconds очищено.
- Анотація останньої застосованої конфігурації більше не містить поля minReadySeconds.

Як оновити об'єкти

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # The annotation contains the updated image to nginx 1.11.9,
    # but does not contain the updated replicas to 2
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
        "metadata":{"annotations":{"name":"nginx-deployment","namespace":"default"},
          "spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}},
            "spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx",
              "ports":[{"containerPort":80}]}]}}}}
        # ...
spec:
  replicas: 2 # Set by `kubectl scale`. Ignored by `kubectl apply`.
  # minReadySeconds cleared by `kubectl apply`
  # ...
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
  spec:
    containers:
      - image: nginx:1.16.1 # Set by `kubectl apply`
        # ...
        name: nginx
        ports:
          - containerPort: 80
        # ...
      # ...
    # ...
  # ...
# ...
```

Як видалити об'єкти

Існує два підходи до видалення об'єктів, якими керує kubectl apply.

Рекомендований: `kubectl delete -f <filename>`

Ручне видалення об'єктів за допомогою імперативної команди є рекомендованим підходом, оскільки це більш чітко визначає, що видаляється, і менш імовірно, що користувач видалить щось ненавмисно:

```
kubectl delete -f <filename>
```

Як видалити об'єкти

Альтернативний: `kubectl apply -f <directory/> --prune -l your=label`

Використовуйте це, лише якщо знаєте, що робите.

Як альтернативу `kubectl delete`, ви можете використовувати `kubectl apply` для ідентифікації об'єктів, які потрібно видалити після того, як їхні конфігураційні файли буде видалено з каталогу.

Застосування з `--prune` запитує сервер API для всіх об'єктів, які відповідають набору міток, і намагається зіставити повернуті конфігурації живих об'єктів із файлами конфігурації об'єктів. Якщо об'єкт відповідає запиту, але він не має файлу конфігурації в каталозі та має анотацію останньої застосованої конфігурації, він видаляється.

```
kubectl apply -f <directory/> --prune -l <labels>
```

Як переглянути об'єкт

Ви можете використовувати `kubectl get з -o yaml` для перегляду конфігурації живого об'єкта:

```
kubectl get -f <filename|url> -o yaml
```

Як застосувати обчислення відмінностей та об'єднання змін

Коли `kubectl apply` оновлює поточну конфігурацію для об'єкта, він надсилає запит на виправлення на сервер API. Патч визначає оновлення для певних полів конфігурації живого об'єкта. Команда `kubectl apply` обчислює цей запит на виправлення, використовуючи файл конфігурації, поточну конфігурацію та анотацію останньої застосованої конфігурації, що зберігається в поточній конфігурації.

Обчислення об'єднувального патча

Команда `kubectl apply` записує вміст файлу конфігурації в анотацію `kubectl.kubernetes.io/last-applied-configuration`. Це використовується для ідентифікації полів, які було видалено з конфігураційного файлу та потребує очищення з поточної конфігурації. Ось кроки, які використовуються для розрахунку того, які поля потрібно видалити або встановити:

1. Обчисліть поля для видалення. Це поля, наявні в останній застосованій конфігурації та відсутні у файлі конфігурації.
2. Обчисліть поля, які потрібно додати або встановити. Це поля у файлі конфігурації, значення яких не відповідають поточній конфігурації.

Як застосувати обчислення відмінностей та об'єднання змін

Ось приклад. Припустимо, це файл конфігурації для об'єкта розгортання:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.16.1 # update the image
          ports:
            - containerPort: 80
```

Як застосувати обчислення відмінностей та об'єднання змін

Крім того, припустімо, що це
поточна конфігурація для того
самого об'єкта розгортання:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # note that the annotation does not contain replicas
    # because it was not updated through apply
    kubectrl.kubernetes.io/last-applied-configuration: |
      {"apiVersion": "apps/v1", "kind": "Deployment",
       "metadata": {"annotations": {}, "name": "nginx-deployment", "namespace": "default"},
       "spec": {"minReadySeconds": 5, "selector": {"matchLabels": {"app": "nginx"}}, "template": {"metadata": {"labels": {"app": "nginx"}},
       "spec": {"containers": [{"image": "nginx:1.14.2", "name": "nginx",
       "ports": [{"containerPort": 80}]}}}}}
    # ...
spec:
  replicas: 2 # written by scale
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
```

Як застосувати обчислення відмінностей та об'єднання змін

Ось обчислення злиття, які виконує `kubectl apply`:

1. Обчисліть поля для видалення, зчитавши значення з останньої застосованої конфігурації та порівнявши їх із значеннями у файлі конфігурації. Очистити поля, для яких явно встановлено значення `null` у файлі конфігурації локального об'єкта, незалежно від того, чи з'являються вони в останній застосованій конфігурації. У цьому прикладі `minReadySeconds` відображається в анотації останньої застосованої конфігурації, але не відображається у файлі конфігурації. **Дія:** видаліть `minReadySeconds` із поточної конфігурації.
2. Обчисліть поля для встановлення, зчитавши значення з файлу конфігурації та порівнявши їх зі значеннями в поточній конфігурації. У цьому прикладі значення `image` у файлі конфігурації не збігається зі значенням у поточній конфігурації. **Дія:** установіть значення зображення в поточній конфігурації.
3. Установіть анотацію останньої застосованої конфігурації відповідно до значення файлу конфігурації.
4. Об'єднайте результати з 1, 2, 3 в один запит на виправлення до сервера API.

Як застосувати обчислення відмінностей та об'єднання змін

Ось поточна конфігурація, яка є
результатом злиття:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # The annotation contains the updated image to nginx 1.11.9,
    # but does not contain the updated replicas to 2
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
        "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
        "spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}},
          "spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx",
            "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  selector:
    matchLabels:
      # ...
      app: nginx
  replicas: 2 # Set by `kubectl scale`. Ignored by `kubectl apply`.
  # minReadySeconds cleared by `kubectl apply`
  # ...
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.16.1 # Set by `kubectl apply`
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
```

Як об'єднуються різні типи полів

Спосіб об'єднання певного поля у файлі конфігурації з поточною конфігурацією залежить від типу поля. Існує кілька типів полів:

- примітивний: поле типу `string`, `integer` або `boolean`. Наприклад, `зображення` та `репліки` є примітивними полями. **Дія:** Замінити.
- `map`, також називається об'єктом: поле типу `map` або складного типу, що містить підполя. Наприклад, `мітки`, `анотації`, `специфікації` та `метадані` – це карти. **Дія:** об'єднати елементи або підполя.
- список: поле, що містить список елементів, які можуть бути примітивними типами або картами. Наприклад, `контейнери`, `порти` та `аргументи` є списками. **Дія:** варіюється.

Коли `kubectl apply` оновлює поле карти чи списку, зазвичай не замінюється все поле, а оновлюються окремі піделементи. Наприклад, під час об'єднання специфікації в розгортанні вся специфікація не замінюється. Натомість підполя специфікації, такі як `репліки`, порівнюються та об'єднуються.

Об'єднання змін до примітивних полів

Примітивні поля замінюються або очищаються.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

Об'єднання змін до полів карти

Поля, які представляють карти, об'єднуються шляхом порівняння кожного з підполів або елементів карти:

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

Об'єднання змін для полів списку типів

Для об'єднання змін до списку використовується одна з трьох стратегій:

- Замініть список, якщо всі його елементи примітивні.
- Об'єднання окремих елементів у список складних елементів.
- Об'єднати список примітивних елементів.

Вибір стратегії здійснюється для кожного поля.

Замініть список, якщо всі його елементи примітивні

Поводьтеся зі списком так само, як із примітивним полем. Замініть або видаліть весь список. Це зберігає порядок.

Приклад: використовуйте `kubectl apply`, щоб оновити поле `args` контейнера в модулі. Це встановлює значення аргументів у поточній конфігурації на значення у файлі конфігурації. Усі елементи `args`, які раніше було додано до поточної конфігурації, буде втрачено. Порядок елементів `args`, визначений у файлі конфігурації, зберігається в поточній конфігурації.

```
# last-applied-configuration value
args: ["a", "b"]

# configuration file value
args: ["a", "c"]

# live configuration
args: ["a", "b", "d"]

# result after merge
args: ["a", "c"]
```

Explanation: The merge used the configuration file value as the new list value.

Об'єднати окремі елементи зі списку складних елементів:

Розглядайте список як карту, а конкретне поле кожного елемента – як ключ. Додайте, видаліть або оновіть окремі елементи. Це не зберігає порядок.

У цій стратегії злиття для кожного поля використовується спеціальний тег, який називається `patchMergeKey.PatchMergeKey` визначено для кожного поля у вихідному коді Kubernetes: **types.go** Під час об'єднання списку карт поле, указане як `patchMergeKey` для певного елемента, використовується як ключ карти для цього елемента.

Приклад: використовуйте `kubectl apply`, щоб оновити поле контейнерів `PodSpec`. Це об'єднує список так, ніби це була карта, де кожен елемент має назву.

Об'єднати окремі елементи зі списку складних елементів:

Пояснення:

- Контейнер під назвою «nginx-helper-a» було видалено, оскільки у файлі конфігурації не було контейнера під назвою «nginx-helper-a».
- Контейнер під назвою «nginx-helper-b» зберіг зміни аргументів у поточній конфігурації. `kubectl apply` зміг визначити, що «nginx-helper-b» у поточній конфігурації був тим самим «nginx-helper-b», що й у файлі конфігурації, навіть якщо їхні поля мали різні значення (немає аргументів у файлі конфігурації). Це тому, що значення поля `patchMergeKey` (ім'я) було ідентичним в обох.
- Контейнер під назвою «nginx-helper-c» було додано, оскільки контейнер із таким ім'ям не з'явився у поточній конфігурації, але контейнер із такою назвою з'явився у файлі конфігурації.
- Контейнер із назвою «nginx-helper-d» було збережено, оскільки в останній застосованій конфігурації не було жодного елемента з такою назвою.

```
# Last-applied-configuration value
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-a # key: nginx-helper-a; will be deleted in result
  image: helper:1.3
- name: nginx-helper-b # key: nginx-helper-b; will be retained
  image: helper:1.3

# configuration file value
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-b
  image: helper:1.3
- name: nginx-helper-c # key: nginx-helper-c; will be added in result
  image: helper:1.3

# Live configuration
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-a
  image: helper:1.3
- name: nginx-helper-b
  image: helper:1.3
  args: ["run"] # Field will be retained
- name: nginx-helper-d # key: nginx-helper-d; will be retained
  image: helper:1.3

# result after merge
containers:
- name: nginx
  image: nginx:1.16
  # Element nginx-helper-a was deleted
- name: nginx-helper-b
  image: helper:1.3
  args: ["run"] # Field was retained
- name: nginx-helper-c # Element was added
  image: helper:1.3
- name: nginx-helper-d # Element was ignored
  image: helper:1.3
```

Значення полів за замовчуванням

Сервер API встановлює для певних полів значення за замовчуванням у поточній конфігурації, якщо вони не вказані під час створення об'єкта.

Ось файл конфігурації для розгортання. У файлі не вказано стратегію:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Значення полів за замовчуванням

Створіть об'єкт за допомогою `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Надрукуйте живу конфігурацію за допомогою `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

Значення полів за замовчуванням

Вихідні дані показують, що сервер API встановлює для кількох полів значення за замовчуванням у поточній конфігурації. Ці поля не були вказані у файлі конфігурації.

```
apiVersion: apps/v1
kind: Deployment
# ...
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  replicas: 1 # defaulted by apiserver
  strategy:
    rollingUpdate: # defaulted by apiserver - derived from strategy.type
    maxSurge: 1
    maxUnavailable: 1
    type: RollingUpdate # defaulted by apiserver
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          imagePullPolicy: IfNotPresent # defaulted by apiserver
          name: nginx
          ports:
            - containerPort: 80
              protocol: TCP # defaulted by apiserver
          resources: {} # defaulted by apiserver
          terminationMessagePath: /dev/termination-log # defaulted by apiserver
          dnsPolicy: ClusterFirst # defaulted by apiserver
          restartPolicy: Always # defaulted by apiserver
          securityContext: {} # defaulted by apiserver
          terminationGracePeriodSeconds: 30 # defaulted by apiserver
# ...
```

Значення полів за замовчуванням

У запиті на виправлення поля за замовчуванням не змінюються за умовчанням, якщо вони явно не очищені в рамках запиту на виправлення. Це може спричинити неочікувану поведінку для полів, які за умовчанням базуються на значеннях інших полів. Коли інші поля буде змінено пізніше, значення за замовчуванням для них не будуть оновлені, якщо їх явно не очищено.

З цієї причини рекомендується, щоб певні поля, стандартні на сервері, були явно визначені у файлі конфігурації, навіть якщо бажані значення відповідають стандартним значенням сервера. Це полегшує розпізнавання конфліктуючих значень, які не будуть повторно встановлені сервером за умовчанням.

```
# Last-applied-configuration
spec:
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80

# configuration file
spec:
  strategy:
    type: Recreate # updated value
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

```
# Live configuration
spec:
  strategy:
    type: RollingUpdate # defaulted value
    rollingUpdate: # defaulted value derived from type
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80

# result after merge - ERROR!
spec:
  strategy:
    type: Recreate # updated value: incompatible with rollingUpdate
    rollingUpdate: # defaulted value: incompatible with "type: Recreate"
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Значення полів за замовчуванням

Пояснення:

1. Користувач створює розгортання, не визначаючи `strategy.type`
2. Сервер за замовчуванням встановлює для `strategy.type` значення `RollingUpdate`, а за замовчуванням — значення `strategy.rollingUpdate`
3. Користувач змінює `strategy.type` на `Recreate`. Значення `strategy.rollingUpdate` залишаються стандартними значеннями, хоча сервер очікує їх очищення. Якби значення `strategy.rollingUpdate` були визначені спочатку у файлі конфігурації, було б зрозуміліше, що їх потрібно видалити.
4. Застосувати не вдається, оскільки `strategy.rollingUpdate` не очищено. Поле `strategy.rollingUpdate` не можна визначити за допомогою `strategy.type Recreate`

Рекомендація: ці поля мають бути явно визначені у файлі конфігурації об'єкта:

- Селектори та мітки PodTemplate для робочих навантажень, таких як Deployment, StatefulSet, Job, DaemonSet, ReplicaSet і ReplicationController
- Стратегія розгортання

Як змінити право власності на поле між файлом конфігурації та прямими імперативними записувачами

Це єдині методи, які слід використовувати для зміни окремого поля об'єкта:

- Використовуйте `kubectl apply`.
- Записуйте безпосередньо в живу конфігурацію, не змінюючи файл конфігурації: наприклад, використовуйте `kubectl scale`.

Зміна власника з прямого імперативного запису на файл конфігурації

Додайте поле до файлу конфігурації. Для поля припиніть прямі оновлення поточної конфігурації, які не проходять через `kubectl apply`.

Зміна власника з конфігураційного файлу на прямий імперативний запис

Починаючи з Kubernetes 1.5, зміна права власності на поле з конфігураційного файлу на імперативний запис вимагає ручних кроків:

- Видаліть поле з конфігураційного файлу.
- Видаліть поле з анотації `kubectl.kubernetes.io/last-applied-configuration` на живому об'єкті.

Зміна методів управління

Об'єктами Kubernetes слід керувати лише одним методом за раз. Перехід від одного методу до іншого можливий, але це ручний процес.

Перехід від імперативного керування командами до декларативної конфігурації об'єктів

Перехід від імперативного керування командами до декларативної конфігурації об'єкта включає кілька кроків вручну:

1. `kubectl get <kind>/<name> -o yaml > <kind>_<name>.yaml` конфігурації:

1. Вручну видалити поле статусу з файлу конфігурації.

2. `kubectl replace --save-config -f <kind>_<name>.yaml` o/last-applied-configuration для об'єкта:

1. Зміна процесів для використання `kubectl apply` для ексклюзивного керування об'єктом

Визначення селекторів контролерів і міток PodTemplate

Рекомендований підхід полягає у визначенні однієї незмінної мітки PodTemplate, яка використовується лише селектором контролера без іншого семантичного значення.

Приклад:

```
selector:
  matchLabels:
    controller-selector: "apps/v1/deployment/nginx"
template:
  metadata:
    labels:
      controller-selector: "apps/v1/deployment/nginx"
```

Питання та відповіді